

The hglm package - a tutorial

[Lars Ronnegard](#)

November 11, 2013

Introduction

This tutorial was written for the half-day H-likelihood course at the Roslin Institute 2013 and gives a brief introduction to analysis of generalized linear models with random effects using the **hglm** package in R.

To learn more about R visit:
<http://www.r-project.org>

0.1 Conventions used in this tutorial

■ Essential code chunks are marked by the red stripe.

Code is written in **Courier-like fonts**.

Code to be written in r console begins with `>`. Comments begin with `#` and `# ^` pertains to the line(s) immediately above the comment.

```
> print("Hello R user!")
[1] "Hello R user!"
> # ^ Prints "Hello R user"
>
> # Another print command
> print("Hello 2")
[1] "Hello 2"
```

Documentation for the hglm package

The hglm package is available on CRAN together with all documentation including a vignette with several examples and theory. To get documentation on a specific function, just type “?” in front of the function name. Try typing `library(hglm); ?hglm`. Thereafter, you can go to the bottom of the documentation page for the hglm function and click on “index” to access all functions, data sets and vignettes in the package.

There are several [YouTube tutorials](#) for the hglm package. There is also a paper describing the package *Ronnegard, Shen & Alam (2010) hglm: A package for fitting hierarchical generalized linear models. [The R Journal](#) 2:20-28.*

Chapter 1

Analysis of binary data with repeated observations

1.1 A first example using the bacteria data in the MASS library

First, we need to load `hglm` and also load the `MASS` library containing the data set `bacteria`. (The `MASS` library is one of the most useful packages in R.) The following R code shows a summary of the contents in `bacteria`. Try running it yourself.

```
> require("hglm")
> # ^ Load hglm
>
> require("MASS")
> # ^ Load MASS
>
> data(bacteria)
> # ^ Load data
> summary(bacteria)
```

y	ap	hilo	week	ID
n: 43	a:124	hi:122	Min. : 0.000	X03 : 5
y:177	p: 96	lo: 98	1st Qu.: 2.000	X04 : 5
			Median : 4.000	X05 : 5
			Mean : 4.455	X07 : 5
			3rd Qu.: 6.000	X08 : 5
			Max. :11.000	X09 : 5
				(Other):190

```

      trt
placebo:96
drug    :62
drug+   :62
```

Now we want to analyze the response variable `y` with `week` as fixed effect and ID

as random (there are repeated observations on each individual). The response variable is the presence of the bacteria *H. influenzae* in children with otitis media in the Northern Territory of Australia. If you want to find out more about the data just type `?bacteria`. The analysis can be performed using the `hglm` package as follows.

```
> g1 <- hglm(fixed = y ~ week,
  random = ~ 1|ID,
  data = bacteria,
  family = binomial(link = logit))
> summary(g1)
Call:
hglm.formula(family = binomial(link = logit), fixed = y ~ week,
  random = ~1 | ID, data = bacteria)

-----
MEAN MODEL
-----

Summary of the fixed effects estimates:

              Estimate Std. Error t-value Pr(>|t|)
(Intercept)  2.30216    0.33627    6.846 9.84e-11 ***
week         -0.13510    0.04127   -3.273  0.00126 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
Note: P-values are based on 193 degrees of freedom

Summary of the random effects estimates:

              Estimate Std. Error
IDX01    0.7472    0.9897
IDX02   -0.2844    0.8385
IDX03    0.8602    0.9591
...
NOTE: to show all the random effects estimates,
use print(summary(hglm.object), print.ranef = TRUE).

-----
DISPERSION MODEL
-----

NOTE: h-likelihood estimates through EQL can be biased.

Dispersion parameter for the mean model:
[1] 0.7581144

Model estimates for the dispersion term:
```

```

Link = log

Effects:
  Estimate Std. Error
    -0.2769    0.1019

Dispersion = 1 is used in Gamma model
on deviances to calculate the standard error(s).

Dispersion parameter for the random effects:
[1] 1.613

Dispersion model for the random effects:

Link = log

Effects:
.|Random1
  Estimate Std. Error
    0.4779    0.2816

Dispersion = 1 is used in Gamma model
on deviances to calculate the standard error(s).

EQL estimation converged in 14 iterations.

```

The `summary` function produces estimates of fixed effects and variance components. The first few random effect estimates (BLUP) are also shown. If you want to see all random effect estimates type `print(summary(g1), print.ranef=TRUE)`. I suggest you try it.

The estimated fixed effects are 2.30216 for the intercept term and -0.13510 for the week effect. The estimated variance component for the random ID effect is 1.613. It is referred to as the "Dispersion parameter for the random effects" because in HGLMs other dispersion parameters than variance components can be fitted.

It is a generalized linear mixed model (GLMM) with binomial outcome that we have fitted. It is a GLMM because the random effect comes from a normal (ie Gaussian) distribution. In a binomial GLMM the dispersion parameter is assumed to be 1 (because the variance in a binomial distribution depends directly on the mean and nothing else). In the fitted `hglm` we do allow this dispersion parameter to be fitted and the estimate is 0.7581144. We could also have fitted the same model with the dispersion parameter fixed to 1: `g1 <- hglm(fixed = y week, random = 1|ID, data = bacteria, family = binomial(link = logit), fix.disp = 1)`. Try it!

Note also that we get a standard error of the variance component for the random effect on a log scale. On the log scale the estimated variance component is 0.4779 with a standard error of 0.23816.

1.2 Using design matrix notation in the hglm package

The formula notation in puts constraints on which models that can be fitted. Therefore, it can be convenient to specify the model in terms of design matrices directly. The following code gives the same results as above.

```
> X <- model.matrix(~bacteria$week)
> #^ Designmatrix for the fixed effects
> y <- 0 + 1*(bacteria$y=="y") # Binary outcome
> Z <- model.matrix(~0 + bacteria$ID)
> #^ Incidence matrix for the random effects
> g2 <- hglm(X=X, y=y, Z=Z,
             family = binomial(link = logit))
> summary(g2)
Call:
hglm.default(X = X, y = y, Z = Z, family = binomial(link = logit))

-----
MEAN MODEL
-----

Summary of the fixed effects estimates:

              Estimate Std. Error t-value Pr(>|t|)
(Intercept)   2.30216    0.33627   6.846 9.84e-11 ***
bacteria$week -0.13510    0.04127  -3.273  0.00126 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
Note: P-values are based on 193 degrees of freedom

Summary of the random effects estimates:

              Estimate Std. Error
bacteria$IDX01   0.7472    0.9897
bacteria$IDX02  -0.2844    0.8385
bacteria$IDX03   0.8602    0.9591
...
NOTE: to show all the random effects estimates,
use print(summary(hglm.object), print.ranef = TRUE).

-----
DISPERSION MODEL
-----

NOTE: h-likelihood estimates through EQL can be biased.

Dispersion parameter for the mean model:
[1] 0.7581144
```

```

Model estimates for the dispersion term:

Link = log

Effects:
      Estimate Std. Error
      -0.2769    0.1019

Dispersion = 1 is used in Gamma model
on deviances to calculate the standard error(s).

Dispersion parameter for the random effects:
[1] 1.613

Dispersion model for the random effects:

Link = log

Effects:
.|Random1
      Estimate Std. Error
      0.4779    0.2816

Dispersion = 1 is used in Gamma model
on deviances to calculate the standard error(s).

EQL estimation converged in 14 iterations.

```

1.3 Random regression using the matrix notation and multiple random effects

As long as we can specify the design matrices, any kind of model can be fitted. Below it is shown how a random regression model can be fitted, where the week effect is fitted as a random slope for each individual.

```

> X <- model.matrix(~bacteria$week)
> #^ Designmatrix for the fixed effects
> y <- 0 + 1*(bacteria$y=="y") # Binary outcome
> Z <- model.matrix(~0 + bacteria$ID)
> #^ Incidence matrix for the random effects
> Z.week <- Z*as.numeric(scale( model.matrix(~0+bacteria$week) ))
> colnames(Z.week) <- 1:50
> #^ Just to have different names in Z.week and Z
> g3 <- hglm(X=X, y=y, Z=cbind(Z, Z.week),
             RandC=c(ncol(Z),ncol(Z.week)),
             family = binomial(link = logit))
> summary(g3)

```

```

Call:
hglm.default(X = X, y = y, Z = cbind(Z, Z.week),
             family = binomial(link = logit),
             RandC = c(ncol(Z), ncol(Z.week)))

-----
MEAN MODEL
-----

Summary of the fixed effects estimates:

              Estimate Std. Error t-value Pr(>|t|)
(Intercept)    2.29715    0.34060   6.744 1.82e-10 ***
bacteria$week -0.13269    0.04326  -3.067  0.00248 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
Note: P-values are based on 189 degrees of freedom

Summary of the random effects estimates:

              Estimate Std. Error
bacteria$IDX01    0.7582    0.9963
bacteria$IDX02   -0.2978    0.8385
bacteria$IDX03    0.8715    0.9661
...
NOTE: to show all the random effects estimates,
use print(summary(hglm.object), print.ranef = TRUE).

Summary of the random effects estimates:

      Estimate Std. Error
1    0.0257    0.3177
2    0.0108    0.3102
3    0.0282    0.3184
...
NOTE: to show all the random effects estimates,
use print(summary(hglm.object), print.ranef = TRUE).

-----
DISPERSION MODEL
-----

NOTE: h-likelihood estimates through EQL can be biased.

Dispersion parameter for the mean model:
[1] 0.739098

Model estimates for the dispersion term:

```



```

Link = log

Effects:
  Estimate Std. Error
    -0.3023    0.1030

Dispersion = 1 is used in Gamma model
on deviances to calculate the standard error(s).

Dispersion parameter for the random effects:
[1] 1.6421 0.1091

Dispersion model for the random effects:

Link = log

Effects:
.|Random1
  Estimate Std. Error
    0.4960    0.2804

.|Random2
  Estimate Std. Error
   -2.2158    0.7198

Dispersion = 1 is used in Gamma model
on deviances to calculate the standard error(s).

EQL estimation converged in 17 iterations.

```

If you wish to plot the estimated random effects, you can extract these values from the hglm object. The values are found in `g3$ranef`.

Chapter 2

Analysis of genetic data - including a relationship matrix

2.1 QTLMAS 2009 data set

In the hglm package there is a data set QTLMAS. The data was simulated for the QTLMAS 2009 workshop in Wageningen, The Netherlands. Here we will analyze the polygenic effects. The data contains the Cholesky factorization of the relationship matrix for 2025 individuals, and there are phenotypic observations from 1000 of these.

```
> data(QTLMAS)
> y <- QTLMAS[,1]
> X <- matrix(1, 1000, 1)
> Z <- as.matrix(QTLMAS[, 2:2026])
> poly1 <- hglm(y=y, X=X, Z=Z)
NOTE: You are fitting a model with one Gaussian random effect term,
      and the number of effects (p) is greater than the number of
      observations (n). Consider turning on the argument 'bigRR' that may
      speed up a lot if p >> n.
> summary(poly1)
Call:
hglm.default(X = X, y = y, Z = Z)

-----
MEAN MODEL
-----

Summary of the fixed effects estimates:

      Estimate Std. Error t-value Pr(>|t|)
X.1    7.2798     0.3107   23.43  <2e-16 ***
```

```

---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
Note: P-values are based on 734 degrees of freedom

Summary of the random effects estimates:

      Estimate Std. Error
Z1  -1.1917      0.5976
Z2   1.6486      0.5401
Z3   1.3194      0.5976
...
NOTE: to show all the random effects estimates,
use print(summary(hglm.object), print.ranef = TRUE).

-----
DISPERSION MODEL
-----

NOTE: h-likelihood estimates through EQL can be biased.

Dispersion parameter for the mean model:
[1] 2.211169

Model estimates for the dispersion term:

Link = log

Effects:
      Estimate Std. Error
      0.7935      0.0522

Dispersion = 1 is used in Gamma model
on deviances to calculate the standard error(s).

Dispersion parameter for the random effects:
[1] 1.503

Dispersion model for the random effects:

Link = log

Effects:
.|Random1
      Estimate Std. Error
      0.4071      0.0868

Dispersion = 1 is used in Gamma model
on deviances to calculate the standard error(s).

EQL estimation converged in 2 iterations.

```

```
> h2 <- poly1$varRanef/(poly1$varRanef + poly1$varFix)
> print(h2)
[1] 0.4045889
```

The residual variance is 2.21 and the additive genetic variance is 1.503, so the heritability is 0.40. The additive relationship matrix can be examined using the code `A <- tcrossprod(Z); image(A)`.

Note also the message about the **bigRR** option. In the model above there are 1000 observations and 2025 additive genetic effects fitted. So the number of levels in the random effect (p) is greater than the number of observations (n). For models with p being more than 10 times larger than n , this option will give much faster computations and produce the same results. Try adding the option **bigRR=TRUE** and check the results.

There is also a **bigRR** package on CRAN specifically developed for analyzing $p \gg n$ problems.

Chapter 3

Advanced topics

3.1 Double HGLM - iterating between several hglm

It is possible to iterate between hglm models to fit a double HGLM, ie a model for the residual variance including random effects.

```
> set.seed(123)
> n=500 #No. of observations
> k=10 #No. of individuals with repeated measurements
> ID <- factor( rep(1:k, each=n/k) )
> #^ Define individuals as a factor. For later use
> Z <- diag(k)%x%rep(1,n/k)
> u <- rnorm(k,0,2) #Random effects in the mean. Simulated variance=4
> u.d <- rnorm(k,0,1) #Random effects in the dispersion. Simulated variance=1
> sigma2e <- exp(Z%*%u.d)
> e <- rnorm(n, 0, sqrt(sigma2e))
> y <- Z%*%u + e
> w.mean <- rep(1,n) #Initial weights
> res.var = 0
> iter = 0
> while (abs(res.var-1) > 0.0001 & iter<20) {
  iter = iter +1
  hg1 <- hglm(X=matrix(1,n,1), y=y, Z=Z, weights=w.mean)
  res.var <- hg1$varFix
  cat("Iteration:", iter,"\n", "Convergence to 1:",res.var, "\n")
  res <- y - hg1$fv[1:n]
  hv <- hg1$hv[1:n]
  dev <- (res^2)/(1-hv)
  hg2 <- hglm(y=dev, X=matrix(1,n,1), Z=Z,
    family=Gamma(link=log), weights=(1-hv)/2)
  w.mean <- 1/hg2$fv
}
Iteration: 1
Convergence to 1: 1.781562
```

```

Iteration: 2
Convergence to 1: 0.9997099
Iteration: 3
Convergence to 1: 1.000001
> cat("Estimated variance components \n")
Estimated variance components
> print(hg1$varRanef)
[1] 3.848661
> print(hg2$varRanef)
[1] 1.129259

```

Note also that the analysis could have been performed directly in the `hglm` function if the random ID effects in the dispersion part were treated as fixed.

```
> hg <- hglm(X=matrix(1,n,1), y=y, Z=Z, X.disp = Z)
```

3.2 Double HGLM - using ASReml-R

If you have access to ASReml and its R interface you can fit the previous DHGLM using the following code.

```

require(asreml)
#DHGLM calculations
mean.w <- c(rep(1,N)); res.var = 0; conv.crit = 0.00001; max.iter = 20
i.iter = 0
while (i.iter<max.iter & abs(res.var-1)>conv.crit) {
  i.iter=i.iter+1
  mean.model <- asreml(y~1,random = ~ID, weights=mean.w)
  res.var <- mean.model$sigma2 #Residual variance
  res <- resid(mean.model, type = "response")
  #Note: strange definition of hat values in ASReml
  asreml.hv <- mean.model$hat
  hv.true <- asreml.hv*mean.w/res.var #Correct hat values
  y_d<-(res^2)/(1-hv.true) #Response for variance model
  var.w <- (1-hv.true)/2 #Weights for variance model
  var.model <- asreml(y_d~1, random = ~ID, weights=var.w,
    family=asreml.Gamma(link=log))
  mean.w <- 1/fitted(var.model) #Update weights for mean model
}

```

3.3 Model selection

To compare models with different variance components, use the adjusted profile likelihood profiled over fixed and random effects (see lecture notes) or the conditional AIC (which can be used for non-nested models as well). These two values are given by

- `summary(hglm.obj)$likelihood$pbvh`
- `summary(hglm.obj)$likelihood$cAIC`

but are only calculated if the option `calc.like=TRUE` is specified.

```
set.seed(123)
n=500 #500 observation
k=10 #10 individuals
Z <- diag(k)%x%rep(1,n/k)
u <- rnorm(k,0,2)
y <- Z%*%u + rnorm(n)
Z2 <- scale(matrix(rnorm(n*20),n,20)) #A random matrix
hg1 <- hglm(y=y, X=matrix(1,n,1), Z=cbind(Z,Z2),
            RandC=c(ncol(Z), ncol(Z2)),
            calc.like=TRUE)
hg2 <- hglm(y=y, X=matrix(1,n,1), Z=Z,
            calc.like=TRUE)
summary(hg1)$likelihood
summary(hg2)$likelihood
```

For this specific simulation, the cAIC and adjusted profile likelihood are more or less the same for both models. So, the model choice is not obvious.

Chapter 4

Advanced modelling using version 2.0 of the hglm package

4.1 Spatial modelling and HL11 correction for Poisson outcomes

The following example is possible to run in the hglm package from version 2.0 (available on CRAN soon). It shows two new options: The **CAR** family for spatial modelling and the option `method = "HL11"` that improves estimation for Poisson outcomes especially for models with $n = p$.

The following code shows analysis of the classical Scottish Lip Cancer data frequently used in spatial modelling.

```
data(cancer)
logE <- log(E)
X11 <- model.matrix(~Paff)
m41 <- hglm(X = X11, y = 0, Z = diag(length(0)),
  family = poisson(), rand.family = CAR(D = nbr),
  offset = logE, conv = 1e-9, maxit = 200, fix.disp = 1, method = "HL11")
summary(m41)
```


Chapter 5

Fitting a large number of random effects using the bigRR package

The **bigRR** package is an extensively developed interface to the **hglm** package for fitting a large number of random effects. Applications of this package for GWAS and genomic predictions was published in *Shen et al. (2013) A novel generalized ridge regression method for quantitative genetics. Genetics 193(4):1255-1268.*

The following example shows the fit for over 200,000 SNPs and a binary trait from Arabidopsis.

```
require(bigRR)
data(Arabidopsis)
X <- matrix(1, length(y), 1)
# fitting SNP-BLUP
SNP.BLUP.result <- bigRR(y = y, X = X, Z = scale(Z),
                        family = binomial(link = 'logit'))
# fitting HEM,
# i.e. with marker-specific shrinkage
HEM.result <- bigRR_update(SNP.BLUP.result, scale(Z),
                        family = binomial(link = 'logit'))
# plot and compare the estimated effects from both methods
split.screen(c(1, 2))
split.screen(c(2, 1), screen = 1)
screen(3)
plot(abs(SNP.BLUP.result$u), cex = .6, col = 'slateblue')
screen(4)
plot(abs(HEM.result$u), cex = .6, col = 'olivedrab')
screen(2)
plot(abs(SNP.BLUP.result$u), abs(HEM.result$u), cex = .6,
     pch = 19, col = 'darkmagenta')
```